

HMS Merge Handover (Patient + Doctor + Admin)

1. Current Scope

This repository implements the ****Patient role**** end-to-end:

- Patient authentication (session-based)
- Dashboard stats
- Appointment booking/cancel/reschedule/list/detail
- Doctor discovery and slot checks
- Prescriptions view
- Data report downloads
- Chat helper

2. Technology Stack

- Backend: Flask, Flask-Login, Flask-SQLAlchemy
- Frontend: Vue 3 + Vite + Axios
- Database: SQLite (current), swappable via 'HMS_DATABASE_URI'
- Auth model: Cookie session (same backend domain)

3. Existing API Surface (Patient)

Auth

- 'POST /api/login'
- 'POST /api/logout'

Dashboard + Appointments

- 'GET /api/dashboard_data'
- 'GET /api/appointments?tab=upcoming|past'
- 'POST /api/appointments'
- 'GET /api/appointments/:id'
- 'POST /api/appointments/:id/cancel'
- 'PUT /api/appointments/:id/reschedule'

Doctors + Slots

- 'GET /api/doctors'
- 'GET /api/doctors/:id/slots?date=YYYY-MM-DD'

Prescriptions

- 'GET /api/prescriptions'
- 'GET /api/prescriptions/:id'

Reports + Chat

- 'GET /api/download_report'

- 'GET /api/download_report_pdf'
- 'GET /api/download_json'
- 'POST /api/chat'

4. Appointment Status Contract (Important for Merge)

Raw DB status remains:

- 'booked'
- 'completed'
- 'cancelled'

API-derived display fields now provided (for UI and merge consistency):

- 'display_status'
- 'display_label'
- 'reschedulable'
- 'cancelable'
- 'status_note'

Derived behavior:

- Completed -> 'Completed'
- Booked + time passed today -> 'Not Attended' (reschedulable)
- Booked + day passed (<2 days) -> 'Not Visited'
- Booked + day passed (>=2 days) -> 'Not Visited Cancelled'

5. Role-Based Integration Model

Recommended top-level role router:

- '/login' -> shared login
- '/patient/*' -> current module
- '/doctor/*' -> doctor module
- '/admin/*' -> admin module

Role resolution flow after login:

1. Auth endpoint returns role metadata ('patient|doctor|admin')
2. Frontend role gateway redirects to role-specific app shell
3. Each role module owns its route namespace and menu

6. Backend Merge Guidelines

1. Keep one Flask app factory or central 'app.py' with blueprints:

- 'patient_bp'
- 'doctor_bp'
- 'admin_bp'

2. Keep shared models in one location:

- 'Patient', 'Doctor', 'Appointment', 'Prescription', 'Department', schedules

3. Add role-aware user identity strategy:

- Option A: separate login tables + role mapping
- Option B: unified users table with role column

4. Keep existing patient endpoints backward-compatible to avoid frontend breaks.

5. Move hardcoded secret/database/cors values to env for all modules.

7. Frontend Merge Guidelines

1. Keep patient UI untouched in 'frontend/src/views/dashboard.vue' behavior.

2. Introduce role shells:

- 'PatientLayout.vue'
- 'DoctorLayout.vue'
- 'AdminLayout.vue'

3. Add route guards by role.

4. Keep shared API client ('api.js') and split role APIs into modules if needed.

5. Preserve existing API response fields used by patient dashboard.

8. Security and Production Notes

- Replace plaintext password check with hash verify before production launch.
- Use HTTPS and set 'HMS_SESSION_SECURE=1'.
- Restrict 'HMS_CORS_ORIGINS' to exact production origins.
- Add database migrations (Alembic/Flask-Migrate) before multi-team merge.
- Add central logging/error middleware for all modules.

9. Handover Checklist

1. Confirm role model and login payload contract.
2. Freeze patient API contract and status fields.
3. Namespace doctor/admin routes without changing patient endpoints.
4. Merge shared models and DB schema migration plan.
5. Add integration test matrix across all 3 roles.
6. Validate session + CORS across deployed domains.
7. Perform UAT for patient flows after merge.

10. What To Share With Team

Share these files:

- 'backend/' and 'frontend/' source

- 'backend/requirements.txt'
- 'backend/.env.example'
- 'README.md'
- 'docs/HMS_Merge_Handover.md'
- 'docs/HMS_Merge_Handover.pdf'